



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
UNIVERSITY OF PIRAEUS

Σύστημα Διαχείρισης Πανεπιστημιακής Βιβλιοθήκης

Μάθημα: Αντικειμενοστρεφής Προγραμματισμός (Java)

1. Εισαγωγή

Η εργασία αυτή υλοποιεί μια εφαρμογή για τη διαχείριση μιας πανεπιστημιακής βιβλιοθήκης σε **Java**. Η εφαρμογή αυτή επιτρέπει την εισαγωγή φοιτητών και καθηγητών, τη διαχείριση υλικού και δανεισμούς και επιστροφές. Στόχος ήταν να εφαρμοστούν στην πράξη οι βασικές αρχές του αντικειμενοστρεφούς προγραμματισμού που διδαχθήκαμε στο μάθημα.

2. Δομή και Ρόλος των Κλάσεων

Το **project** χωρίζεται σε κλάσεις με ξεκάθαρες ευθύνες, ώστε κάθε κλάση να κάνει μία δουλειά και να μην "ξέρει" πράγματα που δεν τη αφορούν.

Η **LibraryItem** είναι **abstract** κλάση που λειτουργεί ως βάση για όλο το υλικό της βιβλιοθήκης. Περιέχει τα κοινά πεδία (*κωδικός, τίτλος, έτος έκδοσης, διαθεσιμότητα*) και την υλοποίηση των μεθόδων **borrowTo()** και **returnItem()**. Οι κλάσεις **Book**, **Magazine** και **DigitalBook** κληρονομούν από αυτήν και προσθέτουν τα δικά τους πεδία. π.χ. το **Book** έχει συγγραφέα και **ISBN**, το **Magazine** αριθμό τεύχους.

Αντίστοιχα, η **Member** είναι **abstract** κλάση για τα μέλη, με υποκλάσεις **StudentMember** και **ProfessorMember**. Η διαφορά τους είναι κυρίως στα επιπλέον στοιχεία (*ΑΜ και τμήμα για τον φοιτητή, ειδικότητα για τον καθηγητή*) και στο όριο δανεισμού (*6 και 15 αντίστοιχα*).

Η **Library** είναι η κεντρική κλάση που κρατά τις λίστες και εκτελεί τη λογική του συστήματος. Η **Loan** αντιπροσωπεύει έναν δανεισμό και κρατά το μέλος, το αντικείμενο και τις ημερομηνίες. Τέλος, η **ReportGenerator** είναι βοηθητική κλάση για εκτύπωση αναφορών.

3. Interfaces

Χρησιμοποιήθηκαν δύο **interfaces**. Το **Borrowable** ορίζει τις μεθόδους **borrowTo()**, **returnItem()** και **isAvailable()** και υλοποιείται από την **LibraryItem**. Με αυτόν τον τρόπο, κάθε τύπος υλικού είναι εγγυημένο ότι έχει αυτή τη συμπεριφορά, χωρίς να χρειάζεται να ελέγχουμε κάθε φορά τον τύπο του.

Το **Searchable** υλοποιείται από την **Library** και ορίζει τις μεθόδους **findItemByCode()** και **findMemberById()**. Ο λόγος που χρησιμοποιήθηκε **interface** εδώ είναι ότι αν στο μέλλον η αναζήτηση υλοποιηθεί διαφορετικά (π.χ. με βάση δεδομένων), η υπόλοιπη εφαρμογή δεν χρειάζεται να αλλάξει.

4. Εφαρμογή Αρχών OOP

Κληρονομικότητα και πολυμορφισμός: Χάρη στην κληρονομικότητα, τα κοινά πεδία και μέθοδοι γράφτηκαν μία φορά στις **abstract** κλάσεις. Η λίστα **ArrayList<LibraryItem>** μπορεί να κρατά ταυτόχρονα βιβλία, περιοδικά και ψηφιακά βιβλία, και η κλήση **item.getDescription()** επιστρέφει κάθε φορά τη σωστή περιγραφή λόγω **overriding**.

Encapsulation: Όλα τα πεδία δηλώθηκαν **private** και η πρόσβαση γίνεται μόνο μέσω **getters/setters**. Οι **setters** περιλαμβάνουν βασικούς ελέγχους: για παράδειγμα, ο τίτλος δεν μπορεί να είναι κενός, το **email** πρέπει να έχει **@**, και το έτος έκδοσης πρέπει να είναι θετικός αριθμός που δεν ξεπερνά το τρέχον έτος.

Modifiers:

- **abstract** στις κλάσεις **LibraryItem** και **Member**, επειδή δεν έχει νόημα να δημιουργηθεί ένα

"γενικό υλικό" ή ένα "γενικό μέλος", πάντα χρειάζεται συγκεκριμένος τύπος.

- **final** στην **ReportGenerator**, επειδή είναι βοηθητική κλάση και δεν έχει λόγο να κληρονομηθεί.
 - **static** στις μεθόδους της **ReportGenerator**, επειδή δεν χρειάζεται να δημιουργηθεί αντικείμενο για να καλέσουμε π.χ. **printActiveLoans()**.
 - **protected** στο πεδίο **isAvailable** της **LibraryItem**, ώστε οι υποκλάσεις να έχουν άμεση πρόσβαση χωρίς να χρειάζεται **getter** μέσα στην ίδια την ιεραρχία.
 - **package-private** χρησιμοποιήθηκε σε βοηθητική μέθοδο εντός του ίδιου **package**, που δεν έχει λόγο να εκτίθεται στον "έξω κόσμο".
-

5. Λειτουργία Μενού

Η **Main** εκκινεί ένα **loop** που εμφανίζει επαναληπτικά το μενού μέχρι ο χρήστης επιλέξει έξοδο (δηλαδή το **0**). Η ανάγνωση γίνεται με **Scanner**. Κάθε επιλογή του μενού καλεί την αντίστοιχη μέθοδο της **Library**. Η **Main** δεν περιέχει λογική, απλώς καλεί αυτό που πρέπει.

Για παράδειγμα, η ροή ενός δανεισμού είναι: αναζήτηση αντικειμένου με κωδικό -> αναζήτηση μέλους με **ID** -> κλήση **borrowTo()** που αλλάζει την κατάσταση διαθεσιμότητας -> δημιουργία νέου **Loan** και προσθήκη στη λίστα.

6. Περιορισμοί/TO-DOS

Τα δεδομένα δεν αποθηκεύονται μόνιμα, χάνονται με κάθε τερματισμό της εφαρμογής. Επίσης, δεν υπάρχει διαχείριση εξαιρέσεων με **try/catch**, οπότε αν ο χρήστης δώσει γράμματα αντί για αριθμό στο μενού, η εφαρμογή θα κρασάρει. Αυτά θα ήταν τα πρώτα πράγματα που θα βελτίωνα σε μια επόμενη έκδοση.